COMP 170 — Lists and Strings Practice

Start with six short exercises, then combine these skills in three larger problems.

Instructions

- Use Python constructs covered in our course materials. Use explicit loops when an exercise asks you to traverse a sequence.
- For Exercises 4–9, write brief pseudocode before coding and give each function type annotations and a docstring.
- For Exercises 7–9, use `main()` to coordinate the program and call it under `if __name__ == "__main__":`. Add helper functions where useful.
- Test the examples and at least one additional case of your own for each coding exercise.

Part A — Quick exercises

1. Same operations, different sequences

Predict each result, then check it in Python.

```
values = [10, 20, 30, 40, 50]
word = "PYTHON"

values[-1]       # ?
values[1:4]      # ?
word[-1]         # ?
word[1:4]        # ?
len(word)        # ?
"TH" in word     # ?
```

Write one slice that returns the last two elements of `values` and one that returns the last two characters of `word`.

2. What actually changes?

Predict what the four print statements display, then run the code.

```
names = ["ada", "ben"]
other = names
copy = names[:]
other[0] = "mia"
copy.append("lee")

word = "ada"
word.upper()

print(names)
print(other)
print(copy)
print(word)
```

a) Explain why `names` changes but `word` does not.
b) Fix the `upper()` line so `word` stores `"ADA"`.
c) Rebuild `word` with its first character replaced by `"I"`, using slicing and concatenation.

3. Edit a waiting list

Start with `waiting = ["Mia", "Omar", "Mia"]`. Use list methods to perform these steps in order:

a) Append `"Lee"`, insert `"Ana"` at the front, remove the first `"Mia"`, then pop the last name into a variable.
b) Print the remaining list and the popped name.
c) Print the number of remaining occurrences of `"Mia"` and the index of `"Omar"` and `"Zoe"`; print `"Not found"` if absent.

4. Repair the readings

Write `repair_readings(values: list[int]) -> int` that replaces every negative value in `values` with 0 **in the original list** and returns how many values were changed. Use an index-based loop.

Example: for `[4, -2, 0, -7]`, the function returns `2` and the list becomes `[4, 0, 0, 0]`. An empty list stays empty and returns `0`.

5. Find the message

Write `get_message(text: str) -> str` that locates the first colon in `text` and returns the text after it, with surrounding whitespace removed (use `strip()`). If there is no colon, return the entire text with any surrounding whitespace removed.

```
"Note:   Bring a notebook  " -> "Bring a notebook"
"  Hello  "                  -> "Hello"
"Time: 3:30"                 -> "3:30"
"Note:   "                   -> ""
```

6. Double each character

Write `double_char(text: str) -> str`. Traverse `text` and build a new string containing two copies of every character, including spaces and punctuation.

```
"Hi!" -> "HHii!!"
"a b" -> "aa  bb"  (two spaces)
""    -> ""
```

Part B — Problem solving

7. Name border — in class

Create `name_border.py`. Write `name_border(full_name: str) -> None` to print the pattern below. Convert the name to uppercase and use `split()` to separate its parts. For each part, print successively shorter suffixes, then successively longer prefixes. Use loops and slices; do not print empty slices.

For `"Ada Li"`, print:

```
ADA
DA
A
A
```

```
AD
ADA
LI
I
L
LI
```

Process all parts, even when the full name contains more than two words. An empty or whitespace-only input prints nothing. In `main()`, ask for a full name and call your function.

Before coding: write the slices that produce the pattern for `"LI"`. Decide what changes from one line to the next.

8. Clean up a club sign-up list

Create `club_list.py`. Write a `main()` function that:

1. Asks for names separated by commas and uses `split(",")` to separate them.
2. Builds a new list of names with surrounding whitespace removed and letters converted to lowercase. Ignore empty entries and duplicates; keep the first occurrence of each name in its original order.
3. Asks for a name to cancel. Remove it if present; otherwise print `"Name not found"`. A blank response skips cancellation. Ignore capitalization and surrounding spaces in this input.
4. Asks for a nonempty partial name and prints all matching names, one per line. Ignore capitalization and surrounding spaces. Use a loop and `find()` to test each name. If none match, print `"No matches"` once.
5. Prints the remaining list in sign-up order, then an alphabetical copy. The alphabetical display must not change the sign-up list.

Choose your own helper functions. The program must also work when no names remain. Do not use sets or dictionaries.

For input `" Mia, OMAR, mia, , Lina, Omar, Leo "`, the cleaned list is:

```
['mia', 'omar', 'lina', 'leo']
```

After cancelling `" OMAR "`, a search for `"I"` prints `mia` and `lina`. The final lists are:

```
Sign-up order: ['mia', 'lina', 'leo']
Alphabetical: ['leo', 'lina', 'mia']
```

9. Caesar cipher — challenge

Create `caesar_cipher.py`. A Caesar cipher shifts letters forward by a key, wrapping from z to a. With key 3, a becomes d and z becomes c.

Write:

- `shift_char(ch: str, key: int) -> str`: Shift one lowercase English letter using `ord()` and `chr()`. Leave other characters unchanged. Assume `ch` has length 1 and `key` is from 0 through 25.
- `encode(message: str, key: int) -> str`: Convert the message to lowercase, traverse it, and call `shift_char()` to build the encoded result. Preserve digits, spaces, and punctuation.

In `main()`, ask for a message and key, then display the encoded message. Assume the key is a valid integer from 0 through 25 and the message contains only English letters, digits, spaces, and punctuation.

```
encode("Hello, World!", 3) -> "khoor, zruog!"
encode("Zoo 123!", 1)     -> "app 123!"
encode("Hi!", 0)          -> "hi!"
```

```
encode("", 4)            -> ""
```

Planning hint: turn a letter into a position from 0 to 25, shift that position, and convert back. How can `%` help you stay within the alphabet?